
PrismMath Tutor: Agentic Retrieval-Augmented Math Tutoring with Local Student Memory and Multi-Context Self-Distillation

Anonymous Author(s)
NLP Capstone Project
your.email@example.com

Abstract

PrismMath Tutor is an agentic mathematics tutoring system that combines retrieval-augmented generation, persistent student memory, and symbolic verification to make language-model tutoring more grounded, adaptive, and auditable. The deployed prototype exposes both a command-line interface and a FastAPI web interface. For each student, the system stores a profile and recent interactions in SQLite, retrieves similar MetaMathQA examples from a local Chroma index on the first turn, prompts an OpenAI-compatible language model with the retrieved examples and student context, and delegates exact symbolic computation to a SymPy-backed verifier when arithmetic or algebraic precision is important. This report documents the framework and positions Adaptive-View Self-Distillation (AVSD) as the finetuning method for training the reasoning model that powers the tutor. AVSD treats full solutions, partial rationales, and final answers as multiple privileged teacher views, then reconstructs token-level supervision by separating cross-view consensus from gated view-specific residual information. We reproduce the AVSD benchmark results on AIME24, AIME25, HMMT25, Codeforces, and LiveCodeBench, where AVSD improves over SFT, GRPO, and single-view on-policy self-distillation. Finally, we describe the PrismMath demonstration workflow, its current engineering limitations, and future work required to integrate a locally fine-tuned AVSD checkpoint into the full tutoring loop.

1 Introduction

Mathematics tutoring is a demanding setting for language-model systems because useful answers must be correct, pedagogically clear, and adapted to the learner’s current state. A model that only emits a final answer is insufficient: students need intermediate reasoning, explanations at the right level of detail, continuity across turns, and a way to avoid small symbolic or arithmetic errors that can invalidate an otherwise useful solution. At the same time, a tutoring demo must be inspectable. When the system uses retrieved examples, a calculator, or stored student context, those components should be visible enough for a developer or evaluator to understand why the tutor answered as it did.

PrismMath Tutor addresses this setting with a compact agentic architecture. The system combines a retrieval-augmented generation (RAG) path over a local MetaMathQA sample, a SQLite-backed student memory, an explicit symbolic verification tool, and a shared chat core used by both the command-line and web interfaces. The implementation is deliberately local-first: the corpus is expected at `data/processed/metamathqa_30k.jsonl`, embeddings are served from `models/qwen3-embedding-4b/`, generated state is stored under `data/` and `logs/`, and inference is routed through an OpenAI-compatible API configured by environment variables. This design makes the demo reproducible on a project machine while keeping sensitive state such as profiles, logs, and API keys out of the source tree.

The central research question for the broader framework is how to make the model behind the tutor better at step-by-step mathematical reasoning. Standard supervised fine-tuning learns from static reasoning traces, but those traces may not match the model’s own rollout distribution. Reinforcement learning with verifiable rewards, including GRPO-style training, can reward correct final answers but often supplies sparse outcome-level feedback [Shao et al., 2024, DeepSeek-AI, 2025]. Distillation provides dense distribution-level supervision [Hinton et al., 2015]; on-policy self-distillation is attractive because it trains on the model’s own trajectories while using a teacher distribution to provide dense token-level learning signals [Agarwal et al., 2024, Zhao et al., 2026]. However, a single privileged teacher context, such as a full reference solution, can be too rigid or can leak information that the student will not observe at test time.

We therefore present Adaptive-View Self-Distillation (AVSD) [Nguyen et al., 2026] as the training method for the PrismMath reasoning model. AVSD uses multiple privileged views of the same training example, such as a full solution, a partial rationale, and a final answer. It first extracts a stable consensus direction across teacher views and then gates the residual support that is specific to one or a few views. This produces a reconstructed target distribution for reverse-KL on-policy self-distillation. The resulting training signal is dense like distillation, on-policy like RL-style rollout training, and more robust than committing to one privileged context.

This report makes three contributions. First, it documents the implemented PrismMath framework, including how RAG, memory, prompt construction, tool dispatch, and logging interact in the local demo. Second, it derives the AVSD objective used as the proposed model-training component, with emphasis on the advantage signal, consensus target, residual gate, and reconstructed loss. Third, it reproduces AVSD benchmark results and connects them to the PrismMath demonstration, clarifying what has been measured by the AVSD paper and what remains a local system demonstration rather than a full end-to-end benchmark.

2 Framework Overview

PrismMath Tutor is organized around a simple principle: the language model should remain responsible for explanation, dialogue, and pedagogical framing, while external components provide context, persistence, and exact computation. The system is not a monolithic prompt. It is a small agentic framework whose behavior is assembled from retrieval, memory, tool use, logging, and a shared chat-turn routine. This section describes the deployed framework before Section 3 describes the AVSD training method for the underlying reasoning model.

2.1 Agentic Components

The current codebase exposes two front ends over the same core logic. The command-line demo starts from `main.py`, delegates to `app.chat_cli`, prompts for a student profile, and then loops over student questions. The web demo is served by `app.web_ui` through FastAPI and renders `app/templates/chat.html`. Both front ends call `run_chat_turn` in `app.chat_core`, which performs the same sequence: load recent memory, optionally retrieve examples, build a student-facing prompt, call the LLM client, parse any tool request, execute the tool, ask the model for a final answer after tool execution, and persist the result.

This organization matters because tutoring failures are often system failures rather than only model failures. If a student asks a question that resembles examples already present in the corpus, the tutor should have access to those examples. If the same student returns for another turn, the tutor should remember recent questions and the preferred explanation style. If the model needs to solve a large equation or verify a fraction, it should not rely only on free-form arithmetic in generated text. The agentic components are therefore designed to solve distinct failure modes.

2.1.1 Retrieval-Augmented Generation

The RAG component addresses the problem of mathematical grounding [Lewis et al., 2020]. A general-purpose language model can often explain algebraic procedures, but it may not remember the exact style, difficulty, or solution patterns of the target corpus. PrismMath uses a local sample of MetaMathQA-style examples as an in-context source. The expected corpus path is `data/processed/metamathqa_30k.jsonl`. Each record must contain at least an identifier, a ques-

tion, and an answer. The loader in `app.rag.dataset` validates these fields and returns a dictionary keyed by example identifier.

Index construction is handled by `app.rag.ingest`. The ingestion routine loads the local examples, encodes their questions with a local Qwen embedding model, and stores the resulting vectors in a persistent ChromaDB collection. The configured collection name is `metamathqa_questions_qwen3_4b`; the index is stored under `data/chroma`. The embedding model is expected under `models/qwen3-embedding-4b/`. This local-model requirement is intentional: retrieval quality should not depend on a remote embedding endpoint changing over time, and the capstone demo can be rebuilt from declared local artifacts.

At runtime, `RAGRetriever` encodes the student's first question, queries ChromaDB for the top- k nearest examples, and returns `RetrievedExample` objects containing the example id, question, answer, and similarity score. The prompt builder then serializes these examples into an in-context block. The examples are not treated as ground truth for the student's exact question. The system prompt explicitly tells the model to use retrieved MetaMathQA examples as in-context examples while answering the current question. This distinction is important: RAG supplies analogies and solution style, not a license to copy an answer for a different problem.

RAG solves three practical problems in PrismMath. First, it gives the tutor examples of mathematical exposition that are close to the student's query. Second, it reduces dependence on model memorization by making local corpus information visible in the prompt. Third, it makes the demo inspectable: the web interface displays the top retrieved examples, allowing evaluators to see whether the retrieval context was relevant. This is particularly useful in an educational setting, where a plausible but irrelevant example can mislead the explanation.

The current implementation retrieves examples only on the first turn of a browser or CLI session. Later turns use conversation memory instead. This is a conservative design choice. First-turn retrieval ensures that the tutor starts with domain-specific support when the student introduces a problem. For follow-up turns, recent interaction history is often more relevant than another independent nearest-neighbor query. The limitation is that a later turn introducing a completely new problem will not automatically trigger retrieval unless the session is restarted or the retrieval policy is extended. A future version should support intent-aware retrieval on any turn that shifts to a new problem.

2.1.2 Student Memory

Student memory addresses personalization and continuity. A tutor should not treat every question as the first interaction with an anonymous user. PrismMath stores profile fields such as student id, nickname, grade, weak areas, strong areas, and preferred explaining style. It also stores previous interactions, retrieved example ids, and tool calls. The memory layer is implemented in `app.memory.store` with SQLite and writes to `data/demo.sqlite3`.

The profile is created or updated at the start of a CLI or web session. In the web interface, form fields are submitted with each question, and `MemoryStore.upsert_profile` persists the latest version. In the CLI, the program first attempts to load an existing profile and only asks for profile fields if no profile exists. This gives the demo a simple but useful form of identity continuity: the same student id can recover prior preferences without requiring a separate authentication system.

Conversation continuity is implemented through recent interaction retrieval. Before each model call, `run_chat_turn` asks the store for the latest interactions for that student. The prompt builder formats them as a recent conversation block. This memory is short-range rather than a full long-term learner model. It is sufficient for continuity across a tutoring session: the model can refer to what the student just asked, avoid repeating the same explanation, and adapt phrasing to the stated preferred style. It also avoids the complexity and privacy risk of storing a broad inferred psychological profile.

Memory also improves auditability. Each interaction stores the query, answer, retrieved example ids, and serialized tool calls. The web and CLI sessions additionally create text logs through `ConversationLogger`. These logs include the student profile, each question, retrieved examples, tool calls, and final answer. For a capstone demonstration, this is valuable because one can inspect a complete turn after the fact: what the student asked, what context the model saw, whether a tool was used, and what answer was returned.

The memory design intentionally separates user data from source code. The README documents deletion commands for removing a single student’s profile, interactions, tool-call rows, retrieved-example rows, and logs, as well as commands for deleting all demo user details. This matters for educational systems because even a prototype can store sensitive learning information. The report therefore treats `data/demo.sqlite3` and `logs/` as generated local artifacts, not as files to be committed or shared.

There are several limitations. The current memory is mostly extractive: it stores recent raw interactions and has a table for summaries, but the active prompt path uses recent turns rather than a learned student model. It does not yet infer mastery, misconception categories, or curriculum state. It also does not implement memory compaction beyond selecting a fixed number of recent interactions. These are reasonable constraints for a prototype. They keep the system transparent and reduce the risk that hidden long-term inferences dominate the tutor’s behavior.

2.1.3 Symbolic Verification Tool

The symbolic verification tool addresses a different failure mode: exactness. Math tutoring systems often fail not because the reasoning plan is wrong, but because a generated intermediate calculation is off by a small amount. PrismMath gives the model a single tool, `verify_answer`, implemented with SymPy [Meurer et al., 2017]. The tool can simplify expressions, solve equations, compare a proposed answer to a symbolic result, and support derivative and integral helper functions. The public prompt contract, however, exposes only the `verify_answer` action.

Tool use begins in the system prompt. The model is instructed to decide internally whether a question is *DIRECT* or *TOOL*. Direct questions can be answered without external computation. Tool questions are those where exact computation, symbolic simplification, equation solving, derivatives, integrals, large arithmetic, or nontrivial fractions would materially improve correctness. If the model chooses tool use, it must output only a fenced block containing JSON with the action and arguments. This strict contract makes the tool call easy to parse and prevents the first response from mixing natural language with a half-formed tool request.

The parser in `app.tools.runtime` searches for a fenced tool block, decodes the JSON, and rejects unsupported tool names. Dispatch is deliberately narrow: only `verify_answer` is accepted. The tool implementation sanitizes input with a restricted character pattern and converts caret notation to Python exponentiation. For equations, it constructs a SymPy equality, solves for the available symbol, and optionally verifies a provided answer. For expressions, it simplifies the expression and optionally compares the simplified result to an expected answer. After tool execution, `run_chat_turn` builds a follow-up prompt containing the original student question, the draft tool request, and the tool result. The model then produces a final student-facing answer.

This two-step pattern supports both correctness and pedagogy. The tool returns a compact symbolic result, but it does not write the explanation. The language model converts that result into a step-by-step answer in the student’s language and preferred style. For example, for the equation

$$987654321x - 123456789 = 555555555,$$

the tool can verify the symbolic solution while the final answer can explain that adding 123456789 to both sides gives $987654321x = 679012344$, and reducing the fraction gives $x = 75445816/109739369$. The tool protects the arithmetic; the model supplies the tutoring.

The tool is intentionally constrained. Its input contract forbids natural language in the problem argument and requires explicit multiplication such as $987654321*x$. This reduces parsing ambiguity and limits unsafe symbolic input. The tradeoff is that the model must learn to produce clean mathematical strings. When it fails, the tool parser or SymPy layer can reject the request, and the tool-call record captures the error. In a production tutor, this tool layer would likely be expanded with structured schemas for equation solving, arithmetic, graphing, and proof checking. For the current prototype, one explicit verifier is enough to demonstrate agentic tool use without obscuring the main system.

2.2 Multi-context Self-Distillation

The framework components above determine how PrismMath behaves at inference time. The remaining question is how to train or adapt the model that performs the reasoning. PrismMath adopts

the view that the reasoning model should learn from multiple training-time contexts, not just from one static answer trace. A math dataset can often provide a full solution, a partial rationale, a final answer, or generated feedback. These views are not all equally useful. A full solution may be informative but can force the model toward a reference reasoning path. A final answer may avoid over-constraining the explanation but gives less token-level guidance. A partial rationale can encourage the early structure of a solution while leaving later steps flexible.

AVSD provides a principled way to combine these views. Instead of choosing one privileged context as the teacher, it evaluates a family of view-conditioned teachers on the same student-generated prefix. Signals that agree across views are treated as stable consensus. Signals that appear only in one or a few views are treated as residual support and are added only when they align with the consensus and remain proportionate. In PrismMath terms, AVSD is the training-side analogue of the runtime architecture: multiple sources of context are useful, but each must be controlled so that it helps the tutor without dominating the final behavior.

3 Fine-tuning Method: Multi-context Self-Distillation

PrismMath’s runtime framework can improve answer grounding and verification, but it cannot by itself make the base model a better mathematical reasoner. The model still needs training signals that reward correct reasoning trajectories and discourage misleading intermediate steps. This section presents Adaptive-View Self-Distillation (AVSD) [Nguyen et al., 2026] as the finetuning method for the model used by PrismMath. The method is especially well matched to a math tutor because training examples naturally come with multiple contexts: a problem statement, a full reference solution, a partial chain of reasoning, and a final answer. These contexts are useful during training but are not all available to the student model at inference time.

3.1 From Single-view Self-Distillation to Token Advantages

Let D be a training set of examples (x, r) , where x is the student-visible prompt and r is privileged training-time information. In a math setting, x can be the problem statement and r can be a reference solution or answer annotation. The student model observes only x during rollout and samples

$$y = (y_1, \dots, y_T) \sim P_\theta^S(\cdot | x).$$

At position t , define the prefix

$$h_t = (x, y_{<t})$$

and the student next-token distribution

$$p_t(v) := P_\theta^S(Y_t = v | h_t), \quad v \in \mathcal{V},$$

where $\mathcal{V}$ is the vocabulary.

In standard single-view self-distillation, the same model is evaluated again in a teacher mode that receives privileged context r . At the same prefix h_t , the teacher distribution is

$$q_t(v) := \text{sg} [P_\theta^T(Y_t = v | h_t, r)],$$

where $\text{sg}[\cdot]$ denotes stop-gradient. The teacher is not a separate larger model; it is the same model under richer conditioning. The local reverse-KL distillation term is

$$\ell_t(\theta) = D_{\text{KL}}(p_t || q_t) = \sum_{v \in \mathcal{V}} p_t(v) (\log p_t(v) - \log q_t(v)).$$

Using $\nabla_\theta p_t(v) = p_t(v) \nabla_\theta \log p_t(v)$, its gradient can be written as

$$\begin{aligned} \nabla_\theta \ell_t &= \sum_{v \in \mathcal{V}} \nabla_\theta p_t(v) (\log p_t(v) - \log q_t(v) + 1) \\ &= \mathbb{E}_{v \sim p_t} [(\log p_t(v) - \log q_t(v) + 1) \nabla_\theta \log p_t(v)]. \end{aligned}$$

The constant 1 is a baseline because

$$\mathbb{E}_{v \sim p_t} [\nabla_\theta \log p_t(v)] = 0.$$

Therefore,

$$\nabla_{\theta} \ell_t = \mathbb{E}_{v \sim p_t} [(\log p_t(v) - \log q_t(v)) \nabla_{\theta} \log p_t(v)].$$

Gradient descent on ℓ_t moves in the direction

$$\mathbb{E}_{v \sim p_t} [A_t(v) \nabla_{\theta} \log p_t(v)],$$

where

$$A_t(v) := \log q_t(v) - \log p_t(v).$$

This is the token-level distillation advantage. If $A_t(v) > 0$, the teacher assigns token v higher probability than the student, so training promotes that token. If $A_t(v) < 0$, training suppresses it. The intuition is close to policy-gradient reinforcement learning, but the reward is a dense teacher-student probability comparison at every token rather than a sparse final-answer reward.

For PrismMath, this is important because many wrong math answers contain both useful and harmful tokens. A sparse verifier might only say that the final answer is incorrect. The reverse-KL advantage can instead identify which generated tokens the privileged teacher would have made more or less likely at each prefix. This dense feedback is valuable for learning step-by-step reasoning.

3.2 Multi-view Privileged Contexts

Single-view self-distillation requires choosing one privileged context. In mathematics, that choice is unstable. A full reference solution gives detailed supervision but may overfit to one derivation. A final answer is compact and avoids forcing a chain of thought, but it gives weak local guidance. A partial rationale can provide useful early structure while leaving room for the student model to complete the solution. AVSD treats these as a teacher family rather than selecting one.

Let the training example contain privileged information r , and let

$$\mathcal{V}(r) = \{r^{(1)}, \dots, r^{(M)}\}$$

be M transformed views, where $r^{(m)} = T_m(r)$. For math finetuning, the natural three-view setting is:

$$r^{(1)} = \text{full solution}, \quad r^{(2)} = \text{partial rationale}, \quad r^{(3)} = \text{final answer}.$$

Each view induces a teacher distribution at the same student prefix:

$$q_t^{(m)}(v) := \text{sg} \left[P_{\theta}^T(Y_t = v \mid h_t, r^{(m)}) \right], \quad m = 1, \dots, M.$$

The per-view token advantage is

$$\Delta_t^{(m)}(v) := \log q_t^{(m)}(v) - \log p_t(v).$$

The central problem is how to combine these advantages. Averaging all teachers too permissively can allow one view to dominate. Requiring all teachers to agree can be too conservative and can discard useful view-specific information. AVSD solves this by decomposing the multi-view signal into consensus and residual components.

3.3 Geometric Consensus Target

The geometric target captures intersection support across teacher views. Define the unnormalized geometric score

$$\tilde{q}_t^G(v) := \left(\prod_{m=1}^M q_t^{(m)}(v) \right)^{1/M} = \exp \left(\frac{1}{M} \sum_{m=1}^M \log q_t^{(m)}(v) \right),$$

and normalize it as

$$q_t^G(v) := \frac{\tilde{q}_t^G(v)}{\sum_{u \in \mathcal{V}} \tilde{q}_t^G(u)}.$$

The geometric mean is conservative. A token receives high support only if it is supported across views. If the full-solution teacher strongly promotes a token but the partial-rationale and answer-only teachers suppress it, the geometric score decreases. This is desirable when the token depends on details that only one privileged view reveals.

For advantage analysis, AVSD uses the unnormalized geometric score because normalization contributes only a token-independent constant. The geometric advantage is

$$\begin{aligned}
A_t^G(v) &:= \log \tilde{q}_t^G(v) - \log p_t(v) \\
&= \frac{1}{M} \sum_{m=1}^M \log q_t^{(m)}(v) - \log p_t(v) \\
&= \frac{1}{M} \sum_{m=1}^M \left(\log q_t^{(m)}(v) - \log p_t(v) \right) \\
&= \frac{1}{M} \sum_{m=1}^M \Delta_t^{(m)}(v).
\end{aligned}$$

Thus the consensus advantage is the average signed update induced by the teacher family. If most views agree that a token should be suppressed, $A_t^G(v)$ is negative. If they agree it should be promoted, $A_t^G(v)$ is positive. For a tutor model, this consensus is the safest part of the training signal because it is less likely to depend on a single privileged annotation format.

3.4 Arithmetic Marginal Target

The arithmetic target captures union support. It is defined as

$$q_t^A(v) := \frac{1}{M} \sum_{m=1}^M q_t^{(m)}(v),$$

with arithmetic advantage

$$A_t^A(v) := \log q_t^A(v) - \log p_t(v).$$

This target is permissive: a token can receive high probability if any view strongly supports it. Such support can be useful. For example, an answer-only view may make a concise algebraic transition more likely, while a full-solution view may favor a detailed explanatory phrase. A good tutor should be able to learn from both signals.

The weakness of the arithmetic target is that it can preserve artifacts. A token may be strongly supported because one privileged view exposes information that the student model cannot infer from the prefix. Directly distilling from q_t^A can therefore encourage tokens whose support is not robust under the student-visible context. In educational terms, the model may learn to imitate a teacher that has already seen the solution rather than learn a reasoning step that follows from the problem.

3.5 Cross-view Residual

The gap between the arithmetic and geometric targets isolates view-specific support. AVSD defines the residual

$$J_t(v) := \log q_t^A(v) - \log \tilde{q}_t^G(v).$$

By the arithmetic-geometric mean inequality, $q_t^A(v) \geq \tilde{q}_t^G(v)$, so

$$J_t(v) \geq 0.$$

The arithmetic advantage can be decomposed as

$$A_t^A(v) = A_t^G(v) + J_t(v).$$

This equation is the key decomposition. $A_t^G(v)$ is the shared signal: it says what the teacher family agrees about. $J_t(v)$ is the additional support retained by the permissive target but absent from strict consensus. A small residual means the views agree. A large residual means at least one view assigns much more support than the others.

The residual is not automatically good or bad. It may contain complementary evidence that improves the model, or it may contain privileged leakage. AVSD therefore does not discard it, but also does not trust it unconditionally. The method keeps the consensus direction and gates the residual contribution.

3.6 Residual Gate

AVSD reconstructs a token-level advantage

$$\hat{A}_t(v) := A_t^G(v) + \lambda_t(v)J_t(v), \quad \lambda_t(v) \in [0, 1].$$

The gate $\lambda_t(v)$ has two components: an alignment component and a magnitude component.

The alignment component checks whether the per-view advantages agree in sign. It is defined as

$$C_t(v) := \frac{|A_t^G(v)|}{\frac{1}{M} \sum_{m=1}^M |\Delta_t^{(m)}(v)| + \epsilon} = \frac{\left| \frac{1}{M} \sum_{m=1}^M \Delta_t^{(m)}(v) \right|}{\frac{1}{M} \sum_{m=1}^M |\Delta_t^{(m)}(v)| + \epsilon}.$$

If the teacher views point in the same direction, the numerator is close to the denominator and $C_t(v)$ is high. If some views promote a token while others suppress it, the signed average can cancel and $C_t(v)$ is low. This distinguishes useful complementary support from direct conflict.

The magnitude component checks whether the residual is proportionate to the consensus:

$$R_t(v) := \frac{|A_t^G(v)|}{|A_t^G(v)| + J_t(v) + \epsilon}.$$

If the residual is much larger than the consensus, then the arithmetic target may override the direction established by the teacher family. In that case $R_t(v)$ becomes small. If the residual is moderate relative to the consensus, $R_t(v)$ allows the residual to adjust the update magnitude.

The final gate is

$$\lambda_t(v) = C_t(v)R_t(v) = C_t(v) \frac{|A_t^G(v)|}{|A_t^G(v)| + J_t(v) + \epsilon}.$$

This gate acts as an adaptive regularizer. It admits view-specific information when the views agree on direction and when the residual is not too large. It suppresses residual information when the views conflict, when consensus is weak, or when one view tries to dominate.

The boundedness property follows directly:

$$\lambda_t(v)J_t(v) = C_t(v) \frac{|A_t^G(v)|J_t(v)}{|A_t^G(v)| + J_t(v) + \epsilon} \leq |A_t^G(v)|.$$

Since $C_t(v) \leq 1$ and the denominator is at least $J_t(v)$, the gated residual cannot exceed the consensus magnitude. Therefore, if $A_t^G(v) < 0$, the reconstructed advantage cannot be flipped to a large positive value solely by a residual from one privileged view. This is the protection against privileged-view leakage: residual support can strengthen a trusted direction, but it cannot freely reverse it.

3.7 Reconstructed Target and AVSD Loss

The reconstructed target corresponding to $\hat{A}_t$ is obtained by reweighting the geometric target with the gated residual and normalizing:

$$q_t^*(v) = \frac{\tilde{q}_t^G(v) \exp(\lambda_t(v)J_t(v))}{\sum_{u \in \mathcal{V}} \tilde{q}_t^G(u) \exp(\lambda_t(u)J_t(u))}.$$

This target interpolates token by token. If $\lambda_t(v) = 0$, the token follows the geometric consensus target. If $\lambda_t(v) = 1$ for all tokens, the method recovers the arithmetic marginal target. In practice, most tokens remain anchored to consensus, and only selected tokens receive residual support.

The final training objective is on-policy reverse-KL distillation to the reconstructed target:

$$\mathcal{L}_{\text{AVSD}}(\theta) = \mathbb{E}_{(x,r) \sim D} \mathbb{E}_{y \sim P_\theta^S(\cdot|x)} \left[\sum_{t=1}^{|y|} D_{\text{KL}}(p_\theta(\cdot | h_t) \| \text{sg}[q_t^*(\cdot)]) \right].$$

Gradients flow through the student distribution only. The teacher views and reconstructed targets are treated as fixed supervision at the current prefix. Equivalently, the sampled-token update uses the reconstructed advantage

$$\hat{A}_t(y_t) = A_t^G(y_t) + \lambda_t(y_t)J_t(y_t).$$

3.8 Training Procedure for PrismMath

In a PrismMath finetuning pipeline, each training iteration would proceed as follows. First, sample a math problem and its available privileged artifacts. Second, generate an on-policy rollout from the current student model using only the problem statement. Third, construct multiple privileged views, such as full solution, partial rationale, and final answer. Fourth, evaluate the same model in teacher mode under each privileged view at the student-generated prefixes. Fifth, compute A_t^G , q_t^A , J_t , λ_t , q_t^* , and the reverse-KL loss. Finally, update the model parameters by minimizing $\mathcal{L}_{\text{AVSD}}$.

The computational overhead is controlled because all teacher views share the same student rollout. The expensive sequential generation path is not repeated for every view; the additional work is view-conditioned forward evaluation at existing prefixes. The AVSD paper reports a modest runtime increase from 41.5 seconds per step for single-view OPSD to 46.7 seconds per step for AVSD under the same Qwen3-4B setup [Nguyen et al., 2026]. For PrismMath, this makes AVSD a plausible training extension rather than an entirely different serving architecture.

The method also matches the runtime philosophy. RAG, memory, and tools each add context to the tutor, but the system controls how that context enters the answer. AVSD applies the same idea during training: privileged contexts are useful, but they must be balanced so that the student learns reasoning behavior available at inference time. The result is a model that can be deployed inside the PrismMath agentic loop with stronger mathematical reasoning before retrieval and tool use are applied.

4 Experiments

This section reports benchmark evidence for the AVSD training method that PrismMath adopts as its model-improvement path. The local PrismMath codebase is a runnable tutoring framework, but it does not yet include a full benchmark harness for evaluating end-to-end tutoring accuracy. Therefore, the quantitative results below are reproduced from the AVSD paper [Nguyen et al., 2026]. They evaluate the finetuning method on math and code reasoning benchmarks and show why AVSD is a strong candidate for training the reasoning model used inside the PrismMath system.

4.1 Evaluation Setting

The AVSD evaluation compares several post-training strategies over three base models: Qwen3-4B, Qwen3-8B, and DeepSeek-R1-Distill-Qwen-7B [Yang et al., 2025, DeepSeek-AI, 2025]. For mathematics, the training data is a mathematical reasoning subset of OpenThoughts, and evaluation uses competition-level benchmarks: AIME 2024, AIME 2025, and HMMT 2025 [Zhang and Math-AI, 2024, 2025, Balunovic et al., 2025]. For code, the training data is drawn from the Python subset of Codeforces, with evaluation on held-out Codeforces problems and LiveCodeBench v6 [Penedo et al., 2025, Jain et al., 2025]. The metric is Avg@8, following the AVSD experimental setup.

The compared methods are:

- **Base**: the original model before task-specific post-training.
- **SFT**: supervised fine-tuning on static reasoning traces.
- **GRPO**: group relative policy optimization with binary outcome rewards.
- **OPSD**: single-view on-policy self-distillation using one privileged teacher context.
- **AVSD**: multi-view self-distillation with gated reconstruction of consensus and residual signals.

4.2 Benchmark Performance

Table 1 shows a consistent pattern. AVSD obtains the best average score for all three base models. On Qwen3-4B, AVSD improves the overall average from 55.0 to 59.9 and outperforms the strongest baseline, OPSD, by 1.7 absolute points across the five reported benchmarks. On Qwen3-8B, AVSD reaches 62.1 average, beating OPSD by 2.9 points and GRPO by 3.0 points. On DeepSeek-R1-Distill-Qwen-7B, AVSD improves the average from 37.7 to 41.4 and outperforms both GRPO and OPSD.

Table 1: Avg@8 accuracy on math and code benchmarks across three base models, reproduced from Nguyen et al. [2026]. CF denotes Codeforces and LCB denotes LiveCodeBench.

Method	AIME24	AIME25	HMMT25	CF	LCB	Avg.
Qwen3-4B	67.5	64.6	40.0	59.8	43.2	55.0
+ SFT	56.3	52.1	29.6	59.1	40.7	47.6
+ GRPO	70.6	65.7	41.8	61.2	43.9	56.6
+ OPSD	73.3	65.4	43.3	63.6	45.2	58.2
+ AVSD (Ours)	76.2	68.3	44.2	65.6	45.4	59.9
Qwen3-8B	71.2	65.4	41.2	60.9	47.6	57.3
+ SFT	70.8	63.5	41.5	61.2	43.2	56.0
+ GRPO	72.5	67.1	43.0	62.8	49.9	59.1
+ OPSD	74.2	65.8	42.5	63.2	50.3	59.2
+ AVSD (Ours)	75.4	69.6	47.1	65.8	52.4	62.1
DeepSeek-R1-Distill-Qwen-7B	48.3	37.9	21.7	44.8	35.7	37.7
+ SFT	48.5	36.2	20.5	41.6	35.2	36.4
+ GRPO	51.2	37.8	23.1	46.2	38.0	39.3
+ OPSD	53.3	37.5	22.6	43.6	37.6	38.9
+ AVSD (Ours)	55.8	39.2	25.4	47.4	39.1	41.4

Table 2: Runtime overhead of AVSD compared with single-view self-distillation, reproduced from Nguyen et al. [2026].

Method	Time / step (s)	Relative time
OPSD	41.5	1.00×
AVSD	46.7	1.13×

The math results are especially relevant to PrismMath. Across AIME24, AIME25, and HMMT25, AVSD improves over the base model for every model family. The gains are not restricted to easier or more familiar benchmarks: HMMT25 improves from 40.0 to 44.2 for Qwen3-4B, from 41.2 to 47.1 for Qwen3-8B, and from 21.7 to 25.4 for DeepSeek-R1-Distill-Qwen-7B. This supports the claim that multi-view distillation improves competition-style reasoning rather than only memorizing a fixed solution format.

The comparison with SFT is also instructive. SFT underperforms the base model in several cases, especially for Qwen3-4B. This is consistent with the risk of off-policy supervised training: static traces can be useful, but they may not align with the model’s own rollout distribution. GRPO improves over the base model more reliably, but it uses sparse final-answer rewards. OPSD provides dense token-level guidance and is stronger than GRPO in several cells, but it still depends on one privileged view. AVSD combines the advantages of on-policy dense guidance with a more robust multi-view target.

4.3 Runtime and View-choice Analysis

Table 2 shows that the additional view-conditioned teacher evaluations add modest overhead in the reported setup. AVSD uses the same student rollouts as single-view OPSD and only adds batched teacher evaluations for multiple privileged contexts. For PrismMath, this matters because the method is intended as a training-time improvement. The serving system does not need to evaluate all privileged views at inference time; it can deploy the resulting fine-tuned model inside the existing RAG, memory, and tool loop.

Table 3 illustrates why selecting a single privileged view is risky. In code, the reference-implementation view performs best among OPSD variants on Codeforces, while the feedback view performs best on LiveCodeBench. AVSD combines all views and obtains the best result on both benchmarks. Although PrismMath is focused on math tutoring, the same principle applies: the best privileged form can vary by task, model, and problem type. A robust training method should use multiple views without allowing any one view to dominate the update.

Table 3: Effect of privileged-view choice in code self-distillation with Qwen3-8B, reproduced from Nguyen et al. [2026].

Method	View(s)	Codeforces	LiveCodeBench	Avg.
Base	–	60.9	47.6	54.3
OPSD	code	63.2	50.3	56.8
OPSD	hint	61.3	48.2	54.8
OPSD	feedback	62.8	50.9	56.9
AVSD (Ours)	all	65.8	52.4	59.1

4.4 Implications for PrismMath

These results do not imply that the local PrismMath prototype has itself achieved the benchmark numbers in Table 1. Rather, they motivate the planned model-training layer. The deployed system already provides retrieval context, student memory, and symbolic verification. AVSD would improve the base reasoning policy before those runtime supports are applied. This separation is useful engineering practice: RAG and tools should not be used to hide a weak reasoning model, and model finetuning should not remove the need for retrieval, personalization, and exact verification.

5 Demonstration

The PrismMath demonstration shows how the implemented system connects model reasoning to retrieval, memory, and tool verification. The demo can be run either through the command line or the web interface. Both modes use the same chat core, so qualitative behavior is consistent across interfaces. This section describes the expected workflow and representative examples from the repository’s `demo_questions.md`.

5.1 Local Setup

The project is designed to run inside its local virtual environment. The README specifies the use of `.\venv\Scripts\python.exe` for package installation and execution. The expected local artifacts are:

```
Corpus          data/processed/metamathqa_30k.jsonl,
Embedding model  models/qwen3-embedding-4b/,
Environment file .env.
```

The environment file supplies `LLM_API_KEY` and `LLM_BASE_URL` for an OpenAI-compatible endpoint. The RAG index is built with

```
python -m app.rag.ingest -reset -batch-size 2,
```

using the project interpreter. If GPU memory is limited, the batch size can be reduced to one. The web interface is launched with Uvicorn and serves the FastAPI app at `http://127.0.0.1:8080`. The CLI demo is launched by running `main.py`.

5.2 End-to-end Turn Flow

A single PrismMath turn proceeds through a fixed sequence. First, the student id is normalized and a profile is created or updated. The profile includes grade, weak areas, strong areas, and preferred explanation style. Second, the system retrieves recent interactions for that student from SQLite. Third, if the current turn is the first turn in the session, the retriever loads the Chroma collection, embeds the question, and selects the top three MetaMathQA examples. Fourth, the prompt builder concatenates the student profile, recent history, retrieved examples, and current question.

The LLM then receives a system prompt that defines the tutor role and the tool-use protocol. The model can answer directly, or it can emit a fenced JSON tool request. If a valid tool request is found, the runtime dispatches it to the SymPy verifier, records the result, and asks the model for a final answer conditioned on the tool output. Finally, the system stores the interaction, retrieved example ids, and tool calls in SQLite, and the session logger writes a human-readable log file under `logs/`.

The web interface makes the agentic components visible. It displays the model answer, the tool call if one was used, the top retrieved in-context examples for the first turn, the current turn number, and the log path. It also renders Markdown and LaTeX through client-side libraries, making mathematical explanations readable in the browser.

5.3 RAG-oriented Questions

Two repository demo questions are drawn from hard examples in the sampled MetaMathQA corpus. One asks about a parametric curve

$$(x, y) = (2 \cos t - \sin t, X \sin t)$$

whose graph can be expressed as

$$ax^2 + bxy + cy^2 = 1,$$

with $X = 64$. The expected ordered triple is

$$(a, b, c) = \left(\frac{1}{4}, \frac{1}{128}, \frac{5}{16384} \right).$$

This example stresses symbolic manipulation and pattern matching to a known style of algebraic solution. When asked as a first turn, PrismMath should retrieve similar MetaMathQA-style examples and expose them as in-context demonstrations. The retrieved examples guide the answer style, while the model still needs to solve the exact current instance.

Another hard question describes a hexagon formed by joining

$$(0, 1), (1, 2), (2, 2), (2, 1), (3, 1), (2, 0), (0, 1)$$

and asks for $a + b + c$ when the perimeter is written as $a + b\sqrt{2} + c\sqrt{5}$. The expected result is

$$3 + 2\sqrt{2} + \sqrt{5},$$

so $a + b + c = 6$. This type of problem benefits from retrieved examples because the solution combines geometry, distance computation, and expression parsing. RAG helps the model remain in the style of contest math explanations, while the final answer remains tied to the actual coordinates in the prompt.

5.4 Tool-oriented Questions

The repository includes a tool-needed equation:

$$987654321x - 123456789 = 555555555.$$

The expected exact solution is obtained by adding 123456789 to both sides:

$$987654321x = 679012344,$$

so

$$x = \frac{679012344}{987654321} = \frac{75445816}{109739369}.$$

This question is useful because the reasoning is simple but the arithmetic is large enough that a language model may make a transcription or reduction error. PrismMath's prompt instructs the model to use `verify_answer` for exact verification, large arithmetic, nontrivial fractions, or equation solving. A clean tool request would use a mathematical string such as

$$987654321*x - 123456789 = 555555555$$

with an empty expected answer if the model only wants the solver result. The final answer should then explain the algebra to the student rather than merely print the tool's dictionary.

Additional demo questions cover a quadratic equation,

$$2x^2 - 7x + 3 = 0,$$

whose solutions are $x = 1/2$ and $x = 3$, and a vertex-form question for

$$f(x) = x^2 - 4x + 1 = (x - 2)^2 - 3.$$

These are medium grade 10–12 questions that should usually be answered directly. They test whether the tutor can explain standard procedures step by step without unnecessary tool calls. The distinction matters: an effective agent should not call a symbolic verifier for every simple task, because unnecessary tool use can slow the interaction and make the explanation feel mechanical.

The demo also includes derivative and large-arithmetic variants. For

$$g(x) = 7x^5 - 3x^2 + 11,$$

the derivative is

$$g'(x) = 35x^4 - 6x.$$

For a rectangular field with dimensions 123456 and 789012, the exact area is

$$123456 \cdot 789012 = 97408265472.$$

These examples test the boundary between direct reasoning and tool-assisted computation. The derivative can be computed by rule, but the system prompt permits verification for symbolic differentiation. The large multiplication should trigger tool use because exact arithmetic is the central task.

5.5 Observed System Properties

The implemented system has several desirable properties for a capstone demo. It is inspectable: retrieved examples and tool calls are surfaced in the UI and stored in logs. It is modular: the LLM client, prompt builder, memory store, retriever, and verifier are separate modules. It is reproducible under local paths: the corpus, Chroma index, SQLite state, logs, and embedding model have explicit locations. It also has clear privacy boundaries for demo data, with README commands for deleting a single student’s details or all local user details.

The system also has clear limitations. RAG is first-turn-only in the current session policy. The symbolic verifier accepts only a constrained mathematical input format and does not parse natural-language problem statements. The active memory path uses recent interactions but does not yet maintain a robust mastery model. Finally, the local repository does not contain a trained AVSD checkpoint or an automated benchmark harness for the PrismMath tutor itself. These limitations are acceptable for the present demonstration, but they define the next engineering steps.

6 Conclusion

PrismMath Tutor demonstrates how a mathematics tutoring system can combine language-model explanation with explicit agentic support. Retrieval supplies local MetaMathQA examples for first-turn grounding. Student memory stores profiles and recent interactions so answers can adapt across turns. A SymPy-backed verifier handles exact symbolic computation when the model identifies a need for tool use. The CLI and FastAPI interfaces share the same chat core, which keeps the demonstration behavior consistent and makes the system easier to inspect.

This report also positions AVSD as the model-training method for the framework. AVSD improves on single-view self-distillation by treating full solutions, partial rationales, and final answers as a family of privileged teacher views. Its objective separates stable cross-view consensus from view-specific residual support, then gates the residual so that privileged information can adjust the update magnitude without freely reversing the consensus direction. The reproduced benchmark results show that AVSD improves over SFT, GRPO, and OPSD across math and code reasoning benchmarks. For PrismMath, the implication is that the deployed tutor should eventually use an AVSD-finetuned reasoning model inside the existing RAG, memory, and tool loop.

Several limitations remain. The current repository is a demonstration system rather than a fully benchmarked tutor. It depends on local corpus files, local Qwen embedding weights, a built Chroma index, and an OpenAI-compatible inference endpoint. RAG currently runs only on the first turn of a session. Memory stores profiles and recent interactions but does not yet infer long-term mastery or misconception state. The symbolic verifier is useful for exact algebra and arithmetic, but its input schema is intentionally narrow and requires clean mathematical strings.

Future work should address these limitations directly. The most important next step is an automated evaluation harness that runs fixed tutoring questions, records retrieved examples, checks final answers

where possible, and measures tool-call appropriateness. The retrieval policy should be extended from first-turn retrieval to intent-aware retrieval whenever a student introduces a new problem. The memory layer should add concise student summaries and misconception tracking while preserving deletion and privacy controls. The tool layer should expand from one symbolic verifier to a typed set of math tools for solving, simplification, plotting, and answer checking. Finally, the AVSD training pipeline should be connected to an actual PrismMath checkpoint so that the system can be evaluated end to end as an agentic tutor powered by a model trained for multi-context mathematical reasoning.

References

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=3zKtaqxLhW>.
- Mislav Balunovic, Jasper Dekoninck, Ivo Petrov, Nikola Jovanovic, and Martin Vechev. Matharena: Evaluating LLMs on uncontaminated math competitions, 2025. URL <https://matharena.ai/>.
- DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=chfJJYC3iL>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, Ondrej Certik, Sergey B. Kirpichev, Matthew Rocklin, Amit Kumar, Sergiu Ivanov, Jason K. Moore, Sartaj Singh, Thilina Rathnayake, Sean Vig, Brian E. Granger, Richard P. Muller, Francesco Bonazzi, Harsh Gupta, Shivam Vats, Fredrik Johansson, Fabian Pedregosa, Matthew J. Curry, Andy R. Terrel, Stepan Roucka, Ashutosh Saboo, Isuru Fernando, Sumith Kulal, Robert Cimrman, and Anthony Scopatz. SymPy: Symbolic computing in python. *PeerJ Computer Science*, 3:e103, 2017.
- Duy Nguyen, Hanqi Xiao, Archiki Prasad, Zaid Khan, Anirban Das, Austin Zhang, Sambit Sahu, Hyunji Lee, Elias Stengel-Eskin, and Mohit Bansal. AVSD: Adaptive-view self-distillation by balancing consensus and teacher-specific privileged signals. *arXiv preprint arXiv:2605.20643*, 2026.
- Guilherme Penedo, Agustin Piqueres Lajarin, Quentin Gallouedec, Nathan Habib, Lewis Tunstall, and Leandro von Werra. Codeforces, 2025. URL <https://huggingface.co/datasets/open-r1/codeforces>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Yifan Zhang and Team Math-AI. American invitational mathematics examination (AIME) 2024, 2024.
- Yifan Zhang and Team Math-AI. American invitational mathematics examination (AIME) 2025, 2025.

Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover.
Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint
arXiv:2601.18734*, 2026.